ClarusLang Lexical Analyzer

Compact Theoretical Report — DFA Execution, Tokenization & Error Trapping

Based on ClarusLang Formal Language Specification v1.0 and Automata3 state-transition diagram.

Scope: The report is theoretical only. It describes DFA states, transitions, token generation, line/column tracking, and lexical error trapping; no executable source code is included.

1. Sample Program & 3-Step Execution Flow

The analyzer processes source from left to right. Whitespace and comments are consumed without tokens; the remaining characters are matched against the DFA and emitted as lexemes.

1.1 Sample ClarusLang Program

```
#activate
int a = 5; // initialize counter
int i;
for (int i = 0; i < 10; i++) {
    if (i % 2 == 0) {
        print(i);
    }
    /* odd values
       are silently skipped */
}
#deactivate
```

Step 1 — Remove Whitespace

At S0, spaces, tabs, and newlines go to WS and produce no token. Newline increments the line counter and resets the column counter.

DFA path: S0 → WS → S0 | action: discard character

```
SOURCE
#activate
int a = 5; // initialize counter
int i;
for (int i = 0; i < 10; i++) {
    if (i % 2 == 0) {
        print(i);
    }
    /* odd values
       are silently skipped */
}
#deactivate

AFTER WHITESPACE REMOVAL
#activate
inta=5;//initializecounter
inti;
for(inti=0;i<10;i++){
if(i%2==0){
print(i);
}/*oddvaluesaresilentlyskipped*/
}
#deactivate
```

The analyzer does not need to physically rewrite the file; this “after” view simply shows which characters are ignored before token formation.

Step 2 — Remove Comments

A slash first enters S10. A second slash selects the single-line branch; a star selects the multi-line branch. Both branches consume characters without emitting tokens.

Single-line: / from S0 → S10 → S11 → S12 … → S0 at newline
Multi-line: /* from S10 → S13 → S14 → S15 → S0 when */ is found

```
AFTER COMMENT REMOVAL
#activate
inta=5;
inti;
for(inti=0;i<10;i++){
if(i%2==0){
print(i);
}
}
#deactivate
```

Step 3 — Character-by-Character Token Generation

The DFA reads one character at a time and follows State × Character Category transitions. Maximal munch is used: continue through the longest valid lexeme, emit it from an accepting state, then rescan the boundary character from S0.

Tracking: line = lexeme start line; column = lexeme start column. Newline changes line/column; every consumed character advances column.

```
Example:  int a = 5;

int : S0 → S52 → S53 → S54(accept)     → DT
 a  : S0 → S1 → S0(accept)             → ID
 =  : S0 → S24(accept)                 → ASSIGN_OP
 5  : S0 → S2 → boundary → NUM
 ;  : S0 → S36(accept)                 → TERMINATOR
```

1.2 Key DFA Transition Groups

Category	Input	Start	State path	Result
Whitespace	space / tab / newline	S0	WS → S0	discard
Identifier	L or _	S0	S1 → S0	ID on boundary
Integer	D	S0	S2	NUM on boundary
Float	D then . then D	S2 → S3	S3	FLOAT_LIT when accepting
String	"	S0	S8 → S9	STRING_LIT when closed
Character	'	S0	S5 → S6 → S7	CHAR_LIT when closed
Comment	/ then /	S0 → S10	S11 → S12	discard to newline
Block comment	/ then *	S0 → S10	S13 → S14 → S15	discard to */
Illegal	undefined category	current	SX	report lexical error

1.3 Step-by-Step State Trace (Representative Dry Run)

The trace below shows the important character transitions from the sample. Repeated comment-body loops are grouped to keep the trace readable while preserving the DFA behavior.

Line No.	Read Char	Start State	Next State	Action / Token Emitted
1	#	S0	S43	start MAIN
1	a	S43	S44	consume
1	c	S44	S45	consume
1	t	S45	S46	consume
1	i	S46	S47	consume
1	v	S47	S48	consume
1	a	S48	S49	consume
1	t	S49	S50	consume
1	e	S50	S51	emit MAIN
2	i	S0	S52	consume int
2	n	S52	S53	consume
2	t	S53	S54	emit DT
2	a	S0	S1	consume ID
2		S1	S0	emit ID; WS
2	=	S0	S24	emit ASSIGN_OP

2	5	S0	S2	consume NUM
2	;	S2	S0	emit NUM
2	;	S0	S36	emit TERMINATOR
2	/	S0	S10	lookahead
2	/	S10	S11	single-line comment
2	…	S11	S12	comment loop
2	\n	S12	S0	discard; line = 3
4	f	S0	S55	consume for
4	o	S55	S56	consume
4	r	S56	S57	emit FOR
4	(	S0	S37	emit LPAREN
4	i	S0	S1	consume ID
4		S1	S0	emit ID; WS
10	/	S0	S10	lookahead
10	*	S10	S13	multi-line comment
10	…	S13	S13	comment loop
10	\n	S13	S13	line increment
11	*	S13	S14	possible close
11	/	S14	S15	close comment
11	EOF/next	S15	S0	resume scanning

1.4 Final Token Stream

Whitespace and comments are absent from the final stream. Token classes follow the ClarusLang specification master token set.

Lex	Class/Token
#activate	MAIN
int	DT
a	ID
=	ASSIGN_OP
5	NUM
;	TERMINATOR
int	DT
i	ID
;	TERMINATOR
for	FOR
(	LPAREN
int	DT
i	ID
=	ASSIGN_OP
0	NUM
;	TERMINATOR
i	ID
<	LT_OP
10	NUM
;	TERMINATOR
i	ID
++	INC_OP
)	RPAREN
{	LBRACE
if	IF
(	LPAREN
i	ID
%	MOD_OP
2	NUM
==	EQ_OP
0	NUM
)	RPAREN
{	LBRACE
print	PRINT
(	LPAREN
i	ID
)	RPAREN
;	TERMINATOR
}	RBRACE
}	RBRACE
#deactivate	END

2. Lexical Error Trapping

When a required transition is unavailable, the DFA reaches SX. The analyzer reports the current line/column and resumes from S0 after discarding the offending input as defined by the error case.

2.1 Invalid / Illegal Character

```
Input:  int total = 5 @ 2;
Trace:  S0 -- @ --> SX
Error:  Lexical Error at Line 2, Column 19: Illegal character '@' — token discarded.
```

A character such as @, $, or a bare # has no valid S0 transition unless # begins the defined program markers.

2.2 Invalid Identifier

```
Input:  int 3abc;
Trace:  3 starts a numeric path; a following letter breaks the valid NUM boundary → SX
Error:  Lexical Error at Line 1, Column 5: Invalid identifier '3abc' — identifiers must begin with a
letter or underscore.
```

The specification requires ID → (L | _) (L | D | _)*. Therefore an identifier cannot begin with a digit.

2.3 Unterminated String / Character Literal

```
Input:  print("hello);
Trace:  S0 -- " --> S8 -- characters --> S8 -- newline/EOF --> SX
Error:  Lexical Error at Line 1, Column 7: Unterminated string literal — expected closing '"'.
```

The same rule applies to character literals: the opening quote must reach the accepting closing-quote state before newline/EOF.

2.4 Unclosed Multi-Line Comment

```
Input:
/* this comment
   is never closed
#deactivate
EOF

Trace:  S0 → S10 → S13 → S13 … → EOF → SX
Error:  Lexical Error at Line 1, Column 1: Unclosed comment — '/*' has no matching '*/' before end of
file.
```

The opening position is retained so the diagnostic points to the unmatched comment opener rather than only the EOF location.

2.5 Exceeding Maximum Token Buffer Length

```
Input:  veryVeryVeryVeryVeryVeryVeryVeryLongIdentifierName;
Trace:  S0 → S1 while the identifier grows → buffer limit exceeded → SX
Error:  Lexical Error at Line 1, Column 1: Token exceeds maximum buffer length.
```

Specification note: The supplied language specification and Automata3 diagram do not define a numeric buffer limit. The exact threshold is therefore an implementation constraint; the report documents the required SX behavior rather than inventing a number.

3. Execution Summary

```
SOURCE TEXT
   ↓
Whitespace → WS → discarded
   ↓
Comments → S11/S12 or S13/S14/S15 → discarded
   ↓
Remaining characters → DFA from S0, left to right
   ↓
Accepting state → emit Lex/Class
   ↓
Undefined transition → SX → report line/column → resume from S0
```

The resulting output is a clean token stream suitable for the next compiler phase. The key lexical guarantees are correct state transitions, maximal-munch token recognition, dynamic source-position tracking, and explicit SX error trapping.